

## PLUGIN DISTRIBUTION

# Constrain plugin dependency versions

Declare version constraints on plugin dependencies so your plugin keeps working when an upstream plugin ships a breaking change.

A plugin can depend on other plugins by listing them in `plugin.json` or in its marketplace entry. By default, a dependency tracks the latest available version, so an upstream release can change the dependency under your plugin without warning. Version constraints let you hold a dependency at a tested version range until you choose to move.

When you install a plugin that declares dependencies, Claude Code resolves and installs them automatically and lists which dependencies were added at the end of the install output. If a dependency later goes missing, `/reload-plugins` and the background plugin auto-update reinstall it, provided its marketplace is already in your configured marketplaces. Re-running `claude plugin install` on the dependent plugin, or adding a marketplace with `claude plugin marketplace add`, also resolves any outstanding missing dependencies. Dependencies from a marketplace you have not added are left unresolved.

This guide is for plugin authors who declare dependencies in `plugin.json` and for marketplace maintainers who tag releases. To install plugins that have dependencies, see [Discover and install plugins](#). For the full manifest schema, see the [Plugins reference](#).

 Dependency version constraints require Claude Code v2.1.110 or later.

## Why constrain dependency versions

Consider an internal marketplace where two teams publish plugins. The platform team maintains `secrets-vault`, an MCP server that wraps a secrets backend. The deploy team maintains `deploy-kit`, which calls `secrets-vault` to fetch credentials during deploys.

`deploy-kit` is tested against `secrets-vault` v2.1.0. Without a version constraint, the next time the platform team tags a release that renames an MCP tool, auto-update moves every engineer's `secrets-vault` to the new version and `deploy-kit` breaks.

With a version constraint, `deploy-kit` declares that it needs `secrets-vault` in the `~2.1.0` range. Engineers with `deploy-kit` installed stay on the highest matching `2.1.x` patch. The deploy team upgrades on their own schedule by publishing a new `deploy-kit` version with a wider constraint.

## Declare a dependency with a version constraint

List dependencies in the `dependencies` array of your plugin's `.claude-plugin/plugin.json`. Each entry is either a plugin name or an object with a version constraint.

The following manifest declares one unversioned dependency and one constrained dependency:

```
.claude-plugin/plugin.json

{
  "name": "deploy-kit",
  "version": "3.1.0",
  "dependencies": [
    "audit-logger",
    { "name": "secrets-vault", "version": "~2.1.0" }
  ]
}
```

An entry can be a bare string with only the plugin name, like `"audit-logger"` in the example above, which depends on whatever version that plugin's marketplace provides. For more control, use an object with these fields:

| Field       | Type   | Description                                                                                                                                                                                                                                                      |
|-------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| name        | string | Plugin name. Resolves within the same marketplace as the declaring plugin. Required.                                                                                                                                                                             |
| version     | string | A <b>semver range</b> such as <code>~2.1.0</code> , <code>^2.0</code> , <code>&gt;=1.4</code> , or <code>=2.1.0</code> . The dependency is fetched at the highest tagged version that satisfies this range.                                                      |
| marketplace | string | A different marketplace to resolve <code>name</code> in. Cross-marketplace dependencies are blocked unless the target marketplace is listed in <u><code>allowCrossMarketplaceDependenciesOn</code></u> in the root marketplace's <code>marketplace.json</code> . |

The `version` field accepts any expression supported by Node's `semver` package, including caret,

tilde, hyphen, and comparator ranges. Pre-release versions such as `2.0.0-beta.1` are excluded unless your range opts in with a pre-release suffix like `^2.0.0-0`.

## Depend on a plugin from another marketplace

By default, Claude Code refuses to auto-install a dependency that lives in a different marketplace than the plugin declaring it. This prevents one marketplace from silently pulling in plugins from a source you have not reviewed.

To allow it, the maintainer of the root marketplace adds the target marketplace name to `allowCrossMarketplaceDependenciesOn` in `marketplace.json`. The root marketplace is the one that hosts the plugin the user is installing; only its allowlist is consulted, so trust does not chain through intermediate marketplaces.

The following `marketplace.json` allows `deploy-kit` to depend on a plugin from `acme-shared`:

`.claude-plugin/marketplace.json`

```
{
  "name": "acme-tools",
  "owner": { "name": "Acme" },
  "allowCrossMarketplaceDependenciesOn": ["acme-shared"],
  "plugins": [
    {
      "name": "deploy-kit",
      "source": "./deploy-kit",
      "dependencies": [
        { "name": "audit-logger", "marketplace": "acme-shared" }
      ]
    }
  ]
}
```

If the field is missing or does not include the target marketplace, install fails with a `cross-marketplace` error naming the field to set. Users can still install the dependency manually first, which satisfies the constraint without changing the allowlist.

## Tag plugin releases for version resolution

Version constraints resolve against git tags on the marketplace repository. For Claude Code to find a

dependency's available versions, the upstream plugin's releases must be tagged using a specific naming convention.

Tag each release as `{plugin-name}--v{version}`, where `{version}` matches the `version` field in that commit's `plugin.json`. From the plugin directory, run:

```
claude plugin tag --push
```

The `claude plugin tag` command derives the tag name from the plugin's manifest and the enclosing marketplace entry. Before creating the tag, it validates the plugin contents, checks that `plugin.json` and the marketplace entry agree on the version, requires a clean working tree under the plugin directory, and refuses if the tag already exists. Add `--dry-run` to see what would be tagged without creating it. Running `git tag secrets-vault--v2.1.0` directly is equivalent if you keep `plugin.json` and the marketplace entry in sync yourself.

The plugin name prefix lets one marketplace repository host multiple plugins with independent version lines. The `--v` separator is parsed as a prefix match on the full plugin name, so plugin names that contain hyphens are handled correctly.

When you install a plugin that declares `{ "name": "secrets-vault", "version": "~2.1.0" }`, Claude Code lists the marketplace's tags, filters to those starting with `secrets-vault--v`, and fetches the highest version satisfying `~2.1.0`. If no matching tag exists, the dependent plugin is disabled with an error listing the available versions.

The resolved tag's semver is recorded separately from `plugin.json`'s `version`, so constraint checks use the tag that was actually fetched even if `plugin.json` at that commit has a stale value. The cache directory name for a tag-resolved install includes a 12-character commit-SHA suffix, so if a maintainer force-moves a tag to a different commit, the next install gets a fresh cache directory instead of reusing stale content.

❗ For `npm` marketplace sources, the constraint does not control which version is fetched, since tag-based resolution applies only to git-backed sources. The constraint is still checked at load time, and the dependent plugin is disabled with `dependency-version-unsatisfied` if the installed version does not satisfy it.

## How constraints interact

When several installed plugins constrain the same dependency, Claude Code intersects their ranges

and resolves the dependency to the highest version that satisfies all of them. The table below shows how common combinations resolve.

| Plugin A requires   | Plugin B requires     | Result                                                                                                      |
|---------------------|-----------------------|-------------------------------------------------------------------------------------------------------------|
| <code>^2.0</code>   | <code>&gt;=2.1</code> | One install at the highest <code>2.x</code> tag at or above <code>2.1.0</code> . Both plugins load.         |
| <code>~2.1</code>   | <code>~3.0</code>     | Install of plugin B fails with <code>range-conflict</code> . Plugin A and the dependency stay as they were. |
| <code>=2.1.0</code> | none                  | The dependency stays at <code>2.1.0</code> . Auto-update skips newer versions while plugin A is installed.  |

Auto-update fetches a constrained dependency at the highest git tag that satisfies every installed plugin's range, rather than at the marketplace's latest version, so the dependency continues to receive updates within its allowed range. If no tag satisfies all ranges, the update is skipped and the skip appears in `/doctor` and the `/plugin` Errors tab, naming the constraining plugin.

When you uninstall the last plugin that constrains a dependency, the dependency is no longer held and resumes tracking its marketplace entry on the next update.

## Enable or disable a plugin with dependencies

Enabling a plugin also enables the plugins it depends on, and disabling a plugin is blocked if another enabled plugin still needs it. Both behaviors require Claude Code v2.1.143 or later. Earlier versions enable or disable only the named plugin and surface a `dependency-unsatisfied` error on the next load.

When you enable a plugin, Claude Code also enables its dependencies at the same scope. If a dependency has its own dependencies, Claude Code enables those too. The success message lists what else was enabled along with the plugin you named. If a dependency can't be enabled, the command refuses and tells you what's blocking and how to fix it:

| Condition                                                    | Result                                                                                              |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| A dependency is not installed                                | Enable fails and prints the <code>claude plugin install</code> command for each missing dependency. |
| A dependency is blocked by your organization's plugin policy | Enable fails and names the blocked dependency.                                                      |

| Condition                                                                                         | Result                                                                                                                            |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| A dependency is set to <code>false</code> at a scope with higher precedence than the target scope | Enable fails. Enable the dependency at that scope, or pass <code>--scope</code> to write there.                                   |
| All dependencies are installed and allowed                                                        | Enable succeeds and writes <code>true</code> for the plugin and each dependency that was not already enabled at the target scope. |

This holds even when a dependency sets `defaultEnabled: false` in its manifest, because Claude Code writes an explicit `true` for it. The same applies at install: a dependency pulled in to satisfy an active plugin installs with `true` regardless of its own default.

When you disable a plugin, Claude Code refuses if another enabled plugin still depends on it. The error names the plugins that depend on it and gives you a chained command that disables them in the right order, ending with the one you asked for.

For example, if `deploy-kit` depends on `secrets-vault`, disabling `secrets-vault` alone fails with output similar to the following:

```
secrets-vault is still required by deploy-kit. Disable that
plugin first, or
disable everything together: claude plugin disable deploy-
kit@acme-tools && claude plugin disable secrets-vault@acme-tools
```

Copy the chained command from the error to disable the full set in one step.

## Remove orphaned auto-installed dependencies

Auto-installed dependencies stay on disk after the plugins that installed them are uninstalled, in case you reinstall a dependent plugin or want to keep using the dependency directly. To clean them up, run `claude plugin prune` to list the auto-installed dependencies that no longer have any installed plugin requiring them and remove them after a confirmation prompt. This requires Claude Code v2.1.121 or later.

```
claude plugin prune
```

By default, prune operates at user scope. Use `--scope project` or `--scope local` to target a different scope. Pass `--dry-run` to list what would be removed without changing anything. Pass `-y` to skip the confirmation prompt. When stdin or stdout is not a terminal, prune lists the orphans

and exits without removing them unless `-y` is passed.

To prune as part of an uninstall, pass `--prune` to `claude plugin uninstall`. After removing the named plugin, Claude Code scans for and removes any auto-installed dependencies that are now orphaned. Plugins you installed yourself are never pruned, only those installed automatically through another plugin's `dependencies` array.

For example, to uninstall `deploy-kit` and clean up the dependencies it leaves behind:

```
claude plugin uninstall deploy-kit --prune
```

## Resolve dependency errors

Dependency problems surface in `claude plugin list`, in the `/plugin` interface, and in `/doctor`. The affected plugin is disabled until you resolve the error. The most common errors and their fixes are listed below.

| Error                                       | Meaning                                                                                                                                                                                                                           | How to resolve                                                                                                                                                                                                                                                                                |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dependency-unsatisfied</code>         | A declared dependency is not installed, or it is installed but disabled.                                                                                                                                                          | Run the <code>claude plugin install</code> command shown in the error message. If the dependency's marketplace is not yet configured, add it with <code>claude plugin marketplace add</code> and Claude Code resolves the dependency automatically. If the dependency is disabled, enable it. |
| <code>range-conflict</code>                 | The version requirements for a dependency cannot be combined. The error message names the cause: no version satisfies all of the ranges, a range is not valid semver syntax, or the combined ranges are too complex to intersect. | Uninstall or update one of the conflicting plugins, fix any invalid <code>version</code> string, simplify long <code>  </code> chains, or ask the upstream author to widen its constraint.                                                                                                    |
| <code>dependency-version-unsatisfied</code> | The installed dependency's version is outside this plugin's declared range.                                                                                                                                                       | Run <code>claude plugin install &lt;dependency&gt;@&lt;marketplace&gt;</code> to re-resolve the dependency against all current constraints.                                                                                                                                                   |
| <code>no-matching-tag</code>                | The dependency's repository has no <code>{name}--v*</code> tag satisfying the range.                                                                                                                                              | Check that the upstream has tagged releases using the convention above, or relax your range.                                                                                                                                                                                                  |

To check for these errors programmatically, run `claude plugin list --json` and read the `errors`

field on each plugin.

## See also

- **Create plugins**: build plugins with skills, agents, and hooks
- **Create and distribute a plugin marketplace**: host plugins for your team
- **Plugins reference**: the full `plugin.json` schema
- **Version management**: how a plugin's own version is resolved and used as the cache key